深圳 / 广州 · 随时到岗 | 2 年开发经验（2024.06 起） | 近一年专职大模型应用与 Agent 工程 | 本科 · 计算机科学与技术（统招）

刘迪君

AI 应用开发 / Agent 工程 · 把大模型约束在确定性工程边界内

我做的事可以概括成一句话：**业务规则写进代码和配置，模型只负责语言理解与结构拼装。** 主导开发的电商经营分析系统已上线、业务每天在用—— 问数一致率从 **35.6%** 提到 **98.1%**，日报制作从人工 2 小时压到 5 分钟内， 50 并发 P95 从 26.6 s 降到 4.18 s，28 个越权用例 100% 拦截。

98.1%
问数一致率（原 35.6%，312 条黄金用例）

65 ms
单次问数 SQL 执行（原 741 ms）

4.18 s
50 并发读路径 P95（原 26.6 s）

100%
28 个越权用例拦截率

01 能力构成

不是「用过很多框架」，而是每一块都能讲清：为什么这么选、边界在哪、怎么验证。

- **Agent 编排**

用代码控制流替代自由 Agent 循环：状态机固定主链路，只在必需处留条件边；会话状态持久化，失败可重放。工具调用、多智能体拆分、MCP 接入。

`LangGraph` `StateGraph` `Checkpointer` `ReAct` `MCP`

- **RAG 与问数**

结构化知识检索（字段元数据 / Few-shot / 别名三类向量）、Join 图补全多表拓扑、粒度对齐防数值放大；指标语义层把口径配置化，模型不直接写 SQL。

`Qdrant` `Elasticsearch` `RRF` `Rerank` `Text-to-SQL`

- **工程底座**

Python AsyncIO 与分层架构、FastAPI / Pydantic / SQLAlchemy；MySQL 8.0 分区裁剪；Redis 缓存键含权限指纹、single-flight 请求合并、分布式锁。

FastAPI MySQL 8.0 Redis Docker Linux

- **安全与质量**

sqlglot AST 七类白名单校验、权限条件下推最内层 WHERE（防推断式越权）、数字白名单防幻觉；638 条用例 + 真实并发压测 + LangFuse 可观测。

sqlglot AST 白名单 篡改检测 pytest LangFuse

02 项目

每个项目三句话说清：解决什么问题、我负责哪部分、实际效果怎么样。

01 工作项目 2025.11 – 2026.07 核心开发 · 5 人团队（后端 2 / 前端 1 / 业务 2） 已上线

面向电商经营场景的智能经营分析 Agent

解决什么问题

业务方问一个数要等数据同学排期；同一指标在不同报表里口径不一致，对数靠人工核对；大模型直接生成 SQL 既不可信、也过不了数据权限。日报长期靠人工拼表。

我负责的部分

核心开发，5 人团队里后端 2 人之一。**独立负责四块**：RAG 检索链路、指标语义层、缓存与并发控制、LangGraph 编排；并落地 AST 白名单与权限下推的安全校验。

实际应用效果

已上线交付业务方日常使用——智能问数、自动日报、异动归因、渠道诊断四类场景跑在真实业务数据上，覆盖运营 / 主管 / 分析师 / 经理四类角色；日报从人工拼表变成系统内直接出。

35.6% → 98.1%

问数一致率（312 条黄金用例，SQL 可执行率 99.4%）

741 → 65 ms

单次问数 SQL 执行（STORED 生成列触发分区裁剪）

120 → 5 min

日报制作（人工 → 系统内）

50% → 80%

异动归因 Top3 命中率（30 个带 ground truth 事件，平均 4.5 s、残差 0）

▾ **展开：架构 · 我负责的四块 · 三次真实故障**



我负责的四块

- **RAG 检索链路**：以字段元数据、Few-shot 用例、指标别名三类向量集合（67 / 104 / 74 条）为检索源，最小检索单元是「一个字段 / 一条用例 / 一个别名」；用 **Join 图 BFS 最小边集**补全多表拓扑，用**粒度对齐**防止一对多 JOIN 把金额放大；以 score_threshold + payload 过滤 + 业务优先级剪枝替代两阶段 rerank，省掉一次模型调用。
- **指标语义层**：12 个核心指标的口径（公式 / 过滤条件 / 时间归属 / 可用维度 / 权限标签）配置化落库，由统一取数规划器翻译成只读 SQL——口径变更不发版，代码侧零硬编码公式。
- **缓存与并发**：缓存键强制包含**权限范围哈希**（否则 A 用户的结果会被 B 用户命中，这是最典型的数据泄露路径）；single-flight 请求合并削峰；分布式锁 SET NX PX 原子获取 + Lua 校验持有者删除 + 长任务按锁时长 1/3 续期；「锁 + 幂等表」双层保证日报推送幂等。
- **多智能体编排**：LangGraph StateGraph 把 15 节点主链路固化为代码控制流（仅 5 处条件边），取代自由 Agent 循环；会话状态经 Checkpointer 持久化。

三次真实故障（比成绩更能说明做事方式）

> **一对多 JOIN 让归因结论被挤出 Top3**
>
> 一次异动归因，系统把主因算到了品类上，与业务判断对不上。根因：品类维度 JOIN 明细表后仍沿用订单级金额求和，一个订单含多商品时金额被重复累加——数字量级「看起来像那么回事」、不报错，只有排名异常才暴露。改为明细金额聚合，并用造数保证「明细金额之和 == 订单实付金额」，实测整月差值 0。

> **权限校验 30 项全绿，而条件注入完全没生效**
>
> 根因是校验脚本自己拼了一遍权限条件再比数字——测试复述了设计意图，没有断言真实路径。重写为 28 个越权用例直接检查权限改写函数的产出，第一次运行就抓出 4 个真实缺陷，最终 100% 拦截。

> **并发压测成功率 0%：async def 里跑同步 I/O**
>
> 声明为 async 的函数内部执行同步阻塞调用，事件循环被拖死——「看起来并发的代码」实际完全不并发。重建并发链路后 P95 从 26.6 s 降到 4.18 s，压测另外抓出 3 个并发缺陷。

一句话说清技术选型

为什么不用自由 Agent	业务链路固定，自由循环换来的是不可复现的失败；固化成状态机后每一次失败都能重放到具体节点。
为什么不让模型写 SQL	模型负责「理解问题 + 拼装结构」，SQL 由语义层编译；模型输出永远不直接进执行路径。
为什么不做 rerank	结构化检索的候选集很小且带业务标签，阈值 + payload 过滤已足够，省一次模型调用的延迟与成本。

`Python` `FastAPI` `LangGraph` `MCP` `MySQL 8.0` `Redis` `Qdrant` `Elasticsearch` `sqlglot` `DeepSeek` `pytest`

02 个人项目 2026.04 – 2026.07 独立开发（后端 + 协议层 + 前端） 已跑通实测

自助式数据看板搭建平台（低代码）

解决什么问题

团队里「要一张看板就得排期开发」，需求积压、响应慢；业务方改一个指标也要走一轮迭代，大量临时需求只能靠手工导表顶着。

我负责的部分

独立开发全栈：物料协议、页面 DSL 与表达式 DSL 解析器、服务端只读 SQL 编译、权限下推、缓存与发布快照，以及前端编辑器，全部由我一个人完成。

实际应用效果

用真实数据结构跑通端到端：受限用户只能看到其权限范围内的数据，无法用「总数 − 可见明细」反推隐藏部分。**平台目前是自用验证版本，尚未在真实团队规模化使用。**

34.9 ms
编译 + 只读校验 + 权限下推 P95

0.18 ms
整页渲染 P50

522 → 0.19 ms
结果缓存；同页 10 并发真实打库 1 次

22 / 22
注入与 AST 逃逸样本 100% 拦截、0 误杀

▾ **展开：五个设计决策与对应实测**

五个设计决策

- **协议驱动渲染**：每类物料声明 props_schema（属性结构 / 数据契约 / 默认值），属性面板自动生成——新增组件只注册物料、不改编辑器，把开闭原则落到代码结构上（8 类物料）。
- **表达式不开放 eval**：表达式来自业务配置，等于把执行权交给配置人员。自研递归下降解析器把它变成自己的 AST，语法里根本没有属性访问、下标、import，所以 `__import__('os').system(...)` 连解析都过不去。
- **权限下推到最内层 WHERE**：不做「查完再过滤」——那样可以用「总数 − 可见明细」反推隐藏数据。条件注入到每个直接读取目标表的 SELECT，并用该层别名限定列名，避免 JOIN 歧义列报错。
- **粒度对齐**：orders ↔ refunds 是 1:N 且非粒度对齐，直接 JOIN 会让 SUM(pay_amount) 按退款条数翻倍；编译器先把退款按 order_id 预聚合到订单粒度，再 LEFT JOIN。
- **发布快照 + 缓存键含权限指纹**：草稿 / 发布双态、发布生成快照可回滚；缓存键强制包含权限范围指纹，避免跨权限命中。

模型在这里的位置

自然语言 → 页面 DSL 只产出**候选**，不做任何直接执行；候选经 AST 白名单校验 + 试渲染双重把关，校验失败把错误信息回灌重新生成。模型能力与确定性边界各自负责一段，出问题能定位到是哪一段。
实测：受限用户只能看到其权限范围内的部分数据，且无法用「总数 − 可见明细」反推隐藏部分。

代码　GitHub · dsl-guard ↗　Gitee 国内镜像 ↗

安全内核（表达式解析器 + SQL 守卫 + 值校验，103 个测试）已于 2026.11 剥离业务代码独立开源；平台本体含业务逻辑，未公开

`Python` `FastAPI` `原生 JavaScript` `MySQL 8.0` `Redis` `sqlglot` `pytest`

03　个人项目　2026.10 – 至今　独立开发　框架已跑通　真模型已实测

Agent 安全靶场：提示注入红队与防御评测

解决什么问题

Agent 一旦带上工具（读文件 / 查库 / 发消息 / 发请求），一段藏在文档里的提示注入就能把数据带出去。但团队没法回答「我们的 Agent 到底有多不安全」——缺一套可复现、可比较的量化口径。

我负责的部分

独立开发全链路：60 条攻击用例设计（5 类攻击面）、带四个工具的目标 Agent、五层可开关防御、规则断言 + LLM 判据、评测流水线与看板。

实际应用效果

60 条攻击用例 + 20 条正常任务，在真实大模型（`deepseek-v4.1-flash-1017`）上全量跑完 160 次：攻击成功率 **5% → 0%**，正常任务通过率 **100% → 95%**。真模型的失效点很集中——它扛得住提示注入，但会跟着文档里的伪造指令去**请求外部或内网地址**，这一层只有工程防御拦得住。后来又用 24 条自适应攻击把基线抬到 **28.3%**，防御全开仍然 **0 漏**，两组的 95% 置信区间不再重叠。

60 + 20

攻击用例 + 正常任务（5 类攻击面各 12 条）

5% → 0%

真模型 ASR：防御全关 → 全开

19 / 20

正常任务完成数（全开配置，无防御误伤）

160

真模型全量运行（22 分钟，逐条落库）

28.3% → 0%

加固版 24 条自适应攻击，每格重复采样 2 轮

13/46 → 0/48

Wilson 区间不重叠，Fisher 双侧 $p < 0.001$

52.2% → 0%

换 qwen3.8-27b 重跑同套用例，防御仍生效

p = 0.0099

② 工具白名单边际，补第 2 轮后达到显著

▾ **展开：为什么做这个 · 评测口径 · 与已有能力的衔接**



为什么值得做

大多数 Agent 作品的展示方式是「它能做什么」；招聘方更想确认的是「它搞不定的时候会怎样」。这个项目把不可见的风险变成可测量的数字，也延续了我前两个项目的同一条主线：**不可信输入不进执行路径**——在 SQL 上是 AST 白名单与权限下推，在 Agent 上是工具权限闸门与轨迹约束。

评测口径（避免自说自话）

- **命中判定**：规则优先——断言工具调用序列（是否调用了越权工具、参数是否含敏感路径、外发请求是否携带密钥）；规则覆盖不到的场景才交给 LLM 判据，判据先用 20 条人工标注样本校准一致性。
- **误杀控制**：同时跑一组不含攻击的正常任务，任何防御导致正常任务失败都计入误杀率，不做「只测攻击、不测正常」的乐观评测。
- **可复现**：固定 seed、温度 0、用例集入库；每次改动只允许动一个变量，跑同一套集对比。

逐层边际（脚本模型基线：去掉某层后 ASR 上升多少）

③ 参数校验 **+21.6 pp**、⑤ 出站过滤 **+8.3 pp**、①②④ 各 **+3.3 pp**。结论是「防御要分层，但真正拦得住的是参数校验」——把路径与 SQL 卡死，比在 prompt 里写一百句「不要听用户的」有用。这一栏最初由确定性模型跑 7 种配置测得（560 次运行）；后来又在真模型上用 24 条加固版攻击集把五层逐一消融了一遍，两组用例集不同、数字分表不混用。

真模型实测到的失效点

60 条用例里真模型只被攻破 3 条，而且全部落在同一处：**工具调用层。**它能识别并拒绝直接的提示注入，但读到夹带伪造指令的文档后，会真的去调 `http_get` 请求 `cdn.shop.example`、`api.shop.example`、甚至 `127.0.0.1:8000/admin` 这类内网管理端口。防御全开后这三条都没能再打穿：T03 / T05 明确命中工具白名单，T08 由参数校验拦下或模型自行拒绝——**模型对齐管得住「说什么」，管不住「调什么」**，这是把安全判断放进防御层而不是塞进 prompt 的直接证据。

把基线抬起来，才能证明防御真的有用

原来那 60 条用例太直白，模型自己就拒掉 57 条，防御全关的 ASR 只有 5%（3/60）——3 条样本算不出统计结论。所以另做了一组 **24 条自适应攻击**：把越权工具调用包装成运维 SOP、变更单、任务重述和等价编码（十进制 / 十六进制 IP、URL 编码、引号表名），每条**重复采样 2 轮**，再对五层防御逐一做消融。结果 **28.3%（13/46）→ 0%（0/48）**，两组 Wilson 95% 区间 [17.3%, 42.5%] 与 [0%, 7.4%] 不再重叠，**Fisher 精确检验双侧 p < 0.001**（原来那组只有 p = 0.244），这时才能说「防御有效」而不是「看起来有效」。判据仍只认工具真的执行成功，不认模型嘴上答应；同一用例两轮结果不一致的也单独列出来。逐层消融里**只有 ② 工具白名单有边际（+13.6 pp，6/44 vs 0/48）**，①③④⑤ 去掉后仍是 0，和脚本模型那组「③ 参数校验是主力」正好相反。第一轮的 `3/23 vs 0/24` 只有 p = 0.109，所以我给 5 个消融配置各补了一轮（120 次调用），合并后 **p = 0.0099，达到显著**——这时才敢把「② 是这组攻击面的关键防线」写进结论。哪一层有用取决于攻击面，不是防御层的固有属性。逐条口径见仓库 `reports/adaptive.md`。

换模型、换载荷、换输入形态，结论稳不稳

补跑三组对照。**换模型**：用 qwen3.8-27b 重跑同一套 24 条，防御全关 ASR 从 28.3% 升到 **52.2%（12/23）**，全开仍是 **0%（0/22）**，Fisher p < 0.001，说明防御不是只对某一个模型有效。**换输入形态**：把伪造的运维指令做成图片送进多模态模型，8 次运行里模型都准确读出了图里的密钥路径与外发邮箱，但都拒绝执行、0 命中（样本仍小，只说明这条链路通了）。**换载荷**：base64、全角同形字、HTML 注释、CSV 公式、分段拼接、双重编码、零宽字符等 8 类编码混淆跑 32 次，基线本身就是 0/16，这组测不出防御贡献，如实记为负面结果。三组各有独立轨迹库与报告：`reports/adaptive-xmodel.md` / `reports/adaptive-mm.md` / `reports/adaptive-v2.md`。

当前进度与口径

真模型全量已跑完：60 条攻击 + 20 条正常 × 2 种配置 = 160 次运行、22 分钟，逐条轨迹落库、报告由脚本生成不接受手填；38 条单测/端到端测试全绿（含 Wilson 区间、Fisher 检验、多模态消息构造的固定值回归），GitHub Actions 在 Python 3.11 / 3.12 上自动跑。

边界清单也在仓库里：消融补轮、跨模型对照、多模态通道、编码载荷实测都已做完，剩下的是 **20 条人工标注**才能启用的 LLM judge、以及本质上无法证明的「绝对安全」与长尾随机性，见 README 7.2 / 7.3 与 `reports/case-readiness.md` / `reports/judge-calibration.md`。

口径声明：真模型那一组用 `deepseek-v4.1-flash-1017` 实测——防御全关 ASR 5.0%（3/60），全开 0.0%（0/60）；正常任务 100% → 95%（19/20），唯一失败那条不是防御拦截造成的，而是模型该轮没产出回答，拦截计数为 0。另一组用确定性脚本模型跑完 7 种配置（ASR 100% → 1.7%，560 次运行约 4 秒），用于验证评测链路与逐层贡献，**不代表真实 LLM 的成功率。**两组分表、不混用。样本量 60 条、每格只采样一次，两组 ASR 的 95% 置信区间互相重叠，所以这里只做现象描述与失效点定位，不把 `5% → 0%` 当作效果显著性的断言；覆盖完整性与无效样本的逐条审计见仓库 `reports/audit-real.md`。

代码 GitHub · dijundev/agent-redteam ↗ Gitee 国内镜像 ↗ 60 条用例、判据与复现命令都在仓库里

`Python` `FastAPI` `LangGraph` `Ollama / Qwen` `SQLite` `原生 JavaScript` `pytest`

附：AI coding 交付实验 —— 键锋打字练习站

用 AI 辅助在 2 小时内从零交付并上线，之后又补了指法诊断、单指专项与节奏反馈。现有 8 种练习模式（含输入法中文）、时间 / 词数两种计时、虚拟键盘指法分区与错键热力图、结果复盘与错字重练。它证明的是「AI 辅助下的交付速度」，不作为 Agent 工程能力的证据，所以放在附录而不是项目区。

打开站点 → 代码

03 五条工程信条

每条都对应一次踩过的坑，不是写给别人看的口号。

01 **模型只做理解与拼装，规则必须落在确定性代码里**

口径、权限、SQL 结构由语义层与 AST 校验承担，模型输出永远不直接进执行路径。这样出错时能定位到「是理解错了，还是规则没覆盖」。

落地：指标语义层（12 指标）、sqlglot 七类校验、权限下推最内层 WHERE。

02 **测试要断言真实路径，不要复述设计**

权限校验 30 项全绿，而真实注入完全没生效——因为脚本自己拼了一遍条件再比数字。测试如果照着设计写，就只是在验证「我以为」。重写后 28 个越权用例第一次运行就抓出 4 个真实缺陷。

03 **粒度不对齐比语法错误危险得多**

JOIN 放大不报错、量级看着合理，只让排名悄悄错掉。修法是造数断言「明细之和 == 订单实付」；低代码平台里同样先用预聚合对齐粒度再 LEFT JOIN。

04 **并发能力必须用真实压测证明**

async def 里跑同步 I/O，压测成功率 0%、P95 26.6 s——「看起来并发」和「真的并发」之间没有中间地带。重建链路后 P95 4.18 s，压测还顺手抓出 3 个并发缺陷。

05 **降级优先于失败**

除配置与元数据库外，任何依赖不可用都不阻止启动：Redis 挂了退本地、Qdrant 不可用退全表元数据、ES 不可用只写库。可用性先于完整性。

04 经历

时间线连续可核对：从数据侧到 Agent 工程，是同一条线往上走。

2026.10 – 至今

个人项目：Agent 安全靶场（提示注入红队与防御评测）

把「不可信输入不进执行路径」这条主线从 SQL 扩展到 Agent 的工具调用链。

2026.04 – 2026.07

个人项目：自助式数据看板搭建平台（低代码）

页面 DSL 与表达式 DSL 自研解析器，服务端编译成只读 SQL；与工作项目共用同一套语义层与权限模型。

2025.11 – 2026.07

智能经营分析 Agent · 核心开发（5 人团队）

RAG 检索链路 / 指标语义层 / 缓存与分布式锁 / LangGraph 编排；交付上线后转线上稳定性保障与运维文档沉淀。

2025.08 – 2025.10

多源异构数据抓取与增量更新框架

统一多来源字段口径、沉淀可复用中间表、维护定时任务集群与运行监控。

2024.06 – 2025.05

数据开发实习 · 元数据与数据字典

盘点表 / 字段 / 指标口径，整理成 YAML 配置解析入库——「口径写在配置里、不写死在代码里」的最早一版实践。

2021.09 – 2025.06

南昌理工学院 · 计算机科学与技术（统招本科）

主修：人工智能、数据挖掘、数据库、MySQL、Python。毕业设计：基于 Python 的小区物业管理系统的设计与实现。

聊聊 Agent 工程?

深圳 · 随时到岗。工作项目代码归属公司，可面谈演示脱敏版本与实测报告；个人项目已开源，代码与复现命令都在仓库里。

邮箱 **liudijunfo@foxmail.com**

GitHub 代码 **agent-redteam**

国内镜像 **Gitee**

靶场看板 在线评测结果